



密码分析学实验报告

姓名: 邱泽辉、江李阳、王健一

学号: 202400460046、202400460042、202400460037

专业: 密码科学与技术

日期: September 23, 2026

Contents

1	实验目的	2
1.1	组员分工	2
2	实验环境	2
3	实验原理	2
3.1	差分路线	2
3.2	差分	2
3.3	CipherFour 算法	3
4	任务一	3
4.1	直接穷举思路	3
4.2	S 盒差分分布表	3
4.3	一轮差分传播	3
4.4	四轮动态规划	5
4.5	高概率差分的搜索	5
5	任务二	6
6	任务三	7
7	实验总结	7
A	实验代码	7

1 实验目的

1.1 组员分工

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows 操作系统、MacOS, Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 差分路线

i 轮差分路线是指对输入对 (X, X') ，在 i 轮加密过程中满足对应第 j 轮的一对输出值 (Y, Y') ($1 \leq j \leq i$)，满足 $\delta X = X \oplus X' = \beta_0$ ， $\delta Y = \oplus Y' = \beta_j$ ($1 \leq j \leq i$)，则称

$$\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i$$

为一条 i 轮差分路线。假设密码算法的输入 X 和各轮密钥 K_i 的取值是相互独立且均匀分布的，对于满足输入差分为 β_0 、输出差分为 β_i (显然满足输入差分为 β_0 的明文不只一对) 的 i 轮差分路线概率为 $DP(\beta_0 \rightarrow \beta_i)$ ，其概率为各轮差分路线概率的乘积

$$DP(\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i) = \prod_{(j=1)}^i DP(\beta_{j-1} \rightarrow \beta_j)$$

注意这里差分路线是精确记录了包括起始、最终结果和中间每一种状态的一条路线。影响上述 i 轮差分路线的概率的主要因素是输入差分非零的 S 盒的个数以及输入差分非零的 S 盒对应的输出差分 (因为当前轮 S 盒的输出即为下一轮 S 盒的输入)，其中输入非零的 S 盒被称为活跃 S 盒。

3.2 差分

i 轮差分是指给定了输入差分 β_0 和 i 轮加密后的输出差分 β_i ，满足上述条件的称为 i 轮差分。与 i 轮差分路线相比，它不指定加密的中间状态。因此，对于 i 轮差分的概率计算如下：假设满足输入差分为 β_0 且 i 轮之后输出差分为 β_i 的 i 轮差分路线共 s 条，则：

$$\begin{aligned} DP(\beta_0 \xrightarrow{i \text{ 轮}} \beta_i) &= \sum_{t=1}^s DP(\beta_0 \xrightarrow{1 \text{ 轮}} \beta_1^t \rightarrow \cdots \beta_{i-1}^t \xrightarrow{1 \text{ 轮}} \beta_i) \\ &= \sum_{t=1}^s \left(\prod_{j=1}^i DP(\beta_{j-1}^t \xrightarrow{1 \text{ 轮}} \beta_j^t) \right) \end{aligned}$$

也就是说 i 轮差分的概率是由所有满足输入和输出条件的 i 轮差分路线共同组成的，它的概率是要大于等于单条差分路线的概率的，而且想要精确计算差分的概率，还是要回到计算每一条差分路线的概率上去。

3.3 CipherFour 算法

4 任务一

本任务要求寻找除题目给定差分之外的高概率四轮差分。题目给出的差分为

$$(0x0, 0x0, 0x0, 0x1) \xrightarrow{4\text{轮}} (0x0, 0x0, 0x0, 0x1),$$

也可以写成

$$0x0001 \xrightarrow{4\text{轮}} 0x0001.$$

4.1 直接穷举思路

对于固定的输入差分 α ，可以遍历所有 2^{16} 个 16 比特明文 x ，构造有序明文对 $(x, x \oplus \alpha)$ ，然后分别进行四轮 CipherFour 加密。若最终密文差分为 β ，则将对对应计数加一。因此，固定输入、输出差分的概率为

$$P(\alpha \rightarrow \beta) = \frac{\#\{x : E(x) \oplus E(x \oplus \alpha) = \beta\}}{2^{16}}.$$

对于每一个非零输入差分，共有 2^{16} 个有序明文对；如果不区分明文对的顺序，则有 2^{15} 个无序明文对。若对全部 $2^{16} - 1$ 个非零输入差分都进行穷举，需要处理约 2^{32} 组明文对，计算开销较大。因此，程序使用 S 盒差分分布表（DDT）对差分进行动态规划。

4.2 S 盒差分分布表

CipherFour 使用一个 4 进 4 出的 S 盒。对于输入差分 α 和输出差分 β ，DDT 定义为

$$\text{DDT}[\alpha][\beta] = \#\{x \in \{0, 1, \dots, 15\} : S(x) \oplus S(x \oplus \alpha) = \beta\}.$$

因此，单个 S 盒的差分传播概率为

$$T[\alpha][\beta] = \frac{\text{DDT}[\alpha][\beta]}{16} = \frac{\text{DDT}[\alpha][\beta]}{2^4}.$$

代码中的 DDT 计算如下：

```
1 for diff_in in range(16):
2     for value in range(16):
3         diff_out = S[value] ^ S[value ^ diff_in]
4         diff_table[diff_in][diff_out] += 1
```

其中，`diff_table[diff_in][diff_out]` 保存输入差分为 `diff_in`、输出差分为 `diff_out` 的出现次数。

4.3 一轮差分传播

一个 16 比特差分可以拆成四个半字节：

$$\Delta = (a_0, a_1, a_2, a_3).$$

若四个 S 盒的输出差分为

$$\Gamma = (b_0, b_1, b_2, b_3),$$

则在固定当前差分的条件下，四个 S 盒同时发生该传播的概率为

$$Q(\Delta, \Gamma) = \prod_{i=0}^3 T[a_i][b_i].$$

例如，输入差分 $0x0001 = (0, 0, 0, 1)$ 时，前三个 S 盒的输入差分为 0，并且

$$T[0][0] = 1.$$

只有最后一个 S 盒产生分支。对于本实验使用的 S 盒，输入差分 1 可以传播为输出差分 1、2、5，其概率分别为 $1/2$ 、 $1/4$ 、 $1/4$ 。因此，经过 S 盒层后可能得到

$$0x0001 (1/2), \quad 0x0002 (1/4), \quad 0x0005 (1/4).$$

代码中的 `score` 保存四个半字节的联合概率分布，其形状为

$$(\text{batch}, 16, 16, 16, 16).$$

处理某一个半字节时，程序计算

$$F_{\text{new}}(\beta) = \sum_{\alpha=0}^{15} F_{\text{old}}(\alpha) T[\alpha][\beta].$$

这对应代码中的矩阵乘法：

```
1 score_matrix = moved_score.reshape(-1, 16)
2 updated_score = score_matrix @ probability_table
```

程序依次处理四个 S 盒：

```
1 for position in range(4):
2     score = update_one_sbox_distribution(
3         score, probability_table, position
4     )
```

对于一个确定的当前 16 比特差分，这一过程等价于四个 S 盒概率的乘积；当当前状态本身是多个差分状态的概率混合时，程序还会对所有前驱状态的概率进行求和，因此需要保存四个半字节的联合分布，而不能只分别保存四个边缘概率。

四个 S 盒处理完成后，将得到的 16 比特差分经过 P 置换。代码预先计算所有 16 比特状态的置换结果：

```
1 permutation = np.array(
2     [P_box(value) for value in range(STATE_SIZE)],
3     dtype=np.int32,
4 )
```

由于 P 是比特置换，满足

$$P(X) \oplus P(Y) = P(X \oplus Y),$$

因此可以直接对差分状态进行 P 置换。经过 P 置换后的差分分布作为下一轮的输入。

4.4 四轮动态规划

设初始输入差分为 α ，经过 r 轮后的差分分布为 $D_r^{(\alpha)}$ 。初始时只有输入差分本身的概率为 1：

$$D_0^{(\alpha)}(\Delta) = \begin{cases} 1, & \Delta = \alpha, \\ 0, & \Delta \neq \alpha. \end{cases}$$

代码通过以下语句完成初始化：

```
1 score[rows, nibble0, nibble1, nibble2, nibble3] = 1.0
```

记一轮四个 S 盒和 P 置换的差分转移概率为 $R(\Delta, \Delta')$ ，则每轮动态规划满足

$$D_{r+1}^{(\alpha)}(\Delta') = \sum_{\Delta} D_r^{(\alpha)}(\Delta) R(\Delta, \Delta').$$

因此，四轮后的输出差分概率为

$$D_4^{(\alpha)}(\beta) = \sum_{\Delta_1, \Delta_2, \Delta_3} R(\alpha, \Delta_1) R(\Delta_1, \Delta_2) R(\Delta_2, \Delta_3) R(\Delta_3, \beta).$$

这个求和包含了从输入差分 α 到输出差分 β 的所有差分路线。代码的外层循环对应四轮传播：

```
1 for _ in range(NUMBER_OF_ROUNDS):
2     for position in range(4):
3         score = update_one_sbox_distribution(
4             score, probability_table, position
5         )
6
7     after_sbox = score.reshape(batch_size, STATE_SIZE)
8     after_pbox[:, permutation] = after_sbox
9     score = after_pbox.reshape(
10         batch_size, 16, 16, 16, 16
11     )
```

这里每一轮的输入和输出都是 16 比特差分状态的概率分布，不是具体的明文或密文。四轮循环结束后，`score` 中保存每个输入差分经过四轮后得到所有输出差分的概率。

4.5 高概率差分的搜索

程序从 `0x0001` 到 `0xFFFF` 遍历全部 $2^{16} - 1$ 个非零输入差分，并将输入差分按 32 个一批进行处理，以降低内存开销。分批处理不会减少搜索范围。

对于每个输入差分 α ，四轮动态规划得到所有输出差分 β 的概率分布。程序使用

```
1 output_differences = np.argmax(
2     output_distribution, axis=1
3 )
```

找到该输入差分对应的最大概率输出差分。随后，再在所有输入差分对应的候选结果中选择全局最大值。

题目要求排除 $0x0001 \rightarrow 0x0001$ ，因此代码只对输入差分 `0x0001` 的输出分布执行：

```

1 candidate_distribution[
2     EXCLUDED_OUTPUT_DIFFERENCE
3 ] = 0.0

```

然后重新选择该输入差分对应的最佳输出差分。

```

PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1> uv run python .\find_best_differential.py
开始遍历全部非零输入差分，并累计各差分路线的概率...
搜索进度: 128/2048 批
搜索进度: 256/2048 批
搜索进度: 384/2048 批
搜索进度: 512/2048 批
搜索进度: 640/2048 批
搜索进度: 768/2048 批
搜索进度: 896/2048 批
搜索进度: 1024/2048 批
搜索进度: 1152/2048 批
搜索进度: 1280/2048 批
搜索进度: 1408/2048 批
搜索进度: 1536/2048 批
搜索进度: 1664/2048 批
搜索进度: 1792/2048 批
搜索进度: 1920/2048 批
搜索进度: 2048/2048 批
CipherFour算法除给定差分外概率最高的4轮差分为:
0x0001 -> 0x0010
4轮差分概率: 0.0479736328125
差分概率的2次幂表示: 2^-4.381614
PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1>

```

Figure 1: 寻找高概率差分

程序最终找到的四轮高概率差分为

$$(0x0, 0x0, 0x0, 0x1) \longrightarrow (0x0, 0x0, 0x1, 0x0),$$

即

$$0x0001 \longrightarrow 0x0010.$$

其基于 DDT 动态规划得到的四轮差分概率为

$$P(0x0001 \rightarrow 0x0010) = 0.0479736328125 \approx 2^{-4.381614}.$$

这里计算的是连接相同输入、输出差分的所有差分路线的概率总和，而不是某一条单独差分路线的概率。后续任务二再通过枚举具体明文对，统计该输入、输出差分在固定 CipherFour 置换网络中的实际路线和频率。

5 任务二

根据任务一找到的新 4 轮高概率差分，要寻找所有对应的差分路线，思路为遍历所有 2^{16} 种输入差分对，即 $x_{in} \in \{0, 1, \dots, 2^{16} - 1\}$ 、 $x' = x \oplus \delta_{in}$ ，记录下 4 轮加密过程中每一轮的中间异或结果，当满足输出差分 $y \oplus y' = \delta_{out}$ 时，该路线即为所要求的 4 轮差分的差分路线，将该路线记录下来，最后输出所有差分路线并计算概率 (注意这里按照输入是有序对时共 2^{16} 种满足条件的差分对，若按照无序对考察，则差分路线和总的差分对数量均减半，但最终概率不变)，计算概率为：

$$P = \frac{\#x : F^4(x) \oplus F^4(x \oplus \delta_{in}) = \delta_{out}}{2^{16}}$$

按照上述思路编写代码，运行结果如下：

```
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> ls

目录: D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2

Mode                LastWriteTime         Length Name
----                -
-a-----         2026/9/23   11:31           3485 find_all_routes.py

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> uv run python .\find_all_routes.py
输入差分: 0x0001
目标输出差分: 0x0010
正在计算CipherFour的一轮S盒与P置换查找表...
满足输入、输出差分的有序明文对数: 3204
不同差分路线数: 6
目标差分频率: 0.04888916015625

差分路线 | 明文对计数 | 路线频率
-----
0x0001 -> 0x0001 -> 0x0001 -> 0x0001 -> 0x0010 | 2048 | 0.0312500000
0x0001 -> 0x0001 -> 0x0010 -> 0x0002 -> 0x0010 | 562 | 0.0089754395
0x0001 -> 0x0101 -> 0x0005 -> 0x0001 -> 0x0010 | 296 | 0.0045166016
0x0001 -> 0x0001 -> 0x0101 -> 0x0005 -> 0x0010 | 238 | 0.0036315918
0x0001 -> 0x0101 -> 0x0105 -> 0x0005 -> 0x0010 | 44 | 0.0006713867
0x0001 -> 0x0101 -> 0x0050 -> 0x0002 -> 0x0010 | 16 | 0.0002441406
路线总频率为: 0.04888916015625
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> |
```

Figure 2: 所有新 4 轮差分路线

6 任务三

7 实验总结

A 实验代码